LAB - DAY 4

Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

Keys, certificates and a look inside TLS



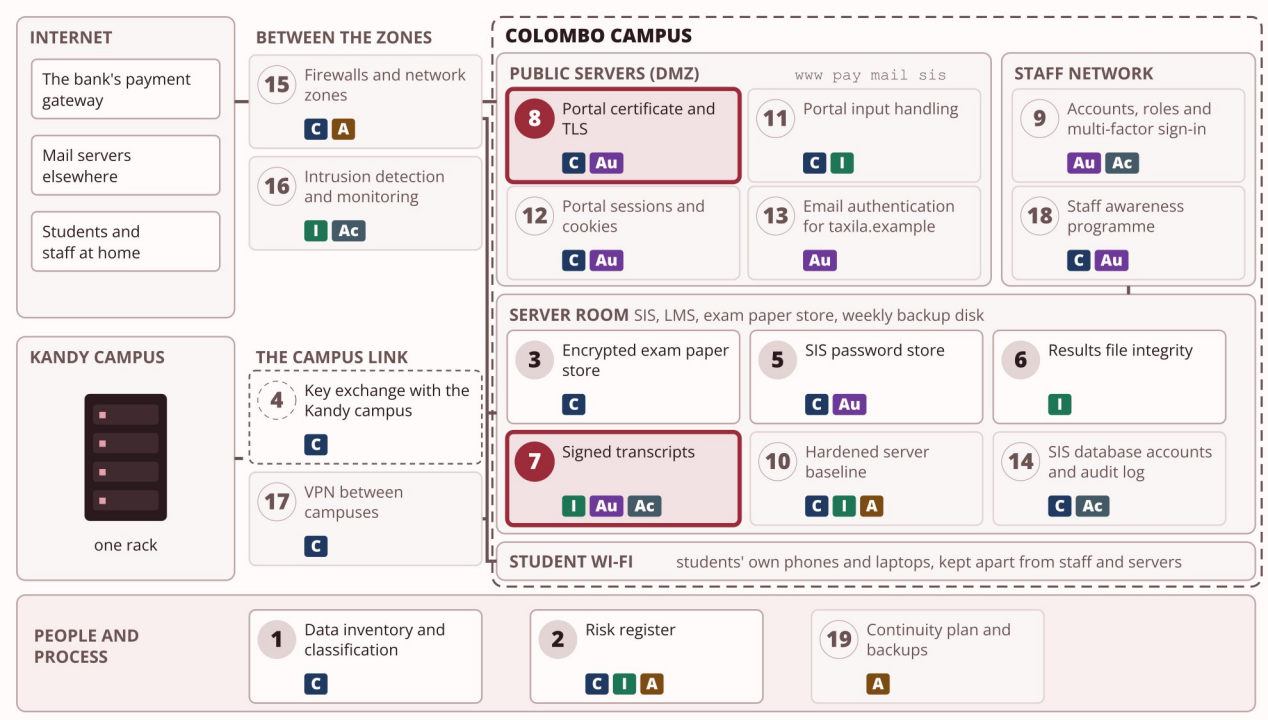
Adds to the register: **7** **8**

Floor tier: every step

Full tier: recommended

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

The security register



TODAY

7 Signed transcripts

8 Portal certificate and TLS

KEY

- Added today
- Added earlier
- Concept only
- Not yet added

IT

HEAD OF IT

Before the portal goes live over secure web, show me whose key it holds, and what a stranger on the network could still read.

The Head of IT, at the start of the lab.

Your personal value and your keys

```
# the signed file holds exactly your index, in capitals
COHNDNE999X000   (14 characters, no new line)
# you submit: the public key, the signature, the check
-----BEGIN PUBLIC KEY-----   ...   Verified OK
# you never submit: the private key
```

Your private key never leaves your laptop. Whoever holds it can sign as you.

SEGMENT 1 - 40 MIN

Generate a key pair and sign your personal file

Both tiers

Sign with the example key first

1 Sign `COHNDNE999X000` with the example private key, then verify.

Expected: Starts `kGcygZsz`, ends `PRBpDQ==`; then `Verified OK`.

Floor path: `RSA Sign` with `SHA-256`, then `To Base64`.

2 Make your own RSA key pair.

Expected: A public block and a private block.

Floor path: `Generate RSA Key Pair`: not the default 1024 bits.

3 Sign your own index, then verify it with your public key (step 4).

Expected: 344 Base64 characters ending in `==`; `Verified OK`.

CyberChef's `RSA Sign` and `RSA Verify` both default to `SHA-1`: set `SHA-256` in both.

The same on the full tier

```
$ printf '%s' COHNDNE999X000 > index.txt
$ openssl genpkey -algorithm RSA -out private.pem
$ openssl pkey -in private.pem -pubout -out public.pem
$ openssl dgst -sha256 -sign private.pem -out index.sig index.txt
$ openssl dgst -sha256 -verify public.pem -signature index.sig index.txt
Verified OK
$ base64 -w0 index.sig > index.sig.b64
```

Use `printf`, not `echo`: `echo` adds a line ending, and you would sign a different file.

SEGMENT 2 - 35 MIN

Make and read a certificate

Both tiers

Make and read a certificate

5 Make or open a self-signed certificate for `sis.taxila.example`.

Expected: A block starting `BEGIN CERTIFICATE`.

Floor path: Open `portal-certificate.txt`.

6 Read subject, issuer, validity, public key, signature.

Expected: Issuer equals subject: self-signed.

Floor path: CyberChef `Parse X.509 certificate`, then `certificate-passport`.

7 Take its SHA-256 fingerprint.

Expected: Provided one: `10:D6:56:4A` … `60:73:A3:81`.

The same on the full tier

```
$ openssl req -x509 -key private.pem -sha256 -days 30 \
    -subj "/CN=sis.taxila.example/O=Taxila College" -out portal-cert.pem
$ openssl x509 -in portal-cert.pem -text -noout
Issuer: CN = sis.taxila.example, O = Taxila College
Subject: CN = sis.taxila.example, O = Taxila College
$ openssl x509 -in portal-cert.pem -noout -fingerprint -sha256
```

With `-x509`, `openssl req` signs the certificate itself instead of writing a signing request.

SEGMENT 3 - 25 MIN

Inspect a live certificate chain

Both tiers

Walk a live chain to its root

8

Open a secure site you use for study and its certificate viewer.

Expected: End-entity, intermediate and root, with a valid padlock.

Floor path: Chrome: Connection is secure, Certificate is valid, Details.

You only read a public certificate that every visitor receives. This is viewing a public site, not attacking it.

SEGMENT 4 - 25 MIN

See what secure web hides

Both tiers

The same form, over plain and secure web

9 Capture a submission over plain web; filter `http`.

Expected: `index=` and your index, readable.

Floor path: `padlock-truth`: the plain-web capture.

10 Capture it over secure web; filter `tls`.

Expected: Only `Application Data`; index not found.

Floor path: `padlock-truth`: the secure-web capture.

```
$ python3 taxila-portal.py
# then, with your own certificate and key:
$ python3 taxila-portal.py --secure --cert portal-cert.pem --key private.pem
```

The warning on your own portal

10 Read the browser's warning on your secure portal.

Expected: Warning: Potential Security Risk Ahead.

Continue only after the fingerprint the browser shows matches the one you took in step 7, and only on a certificate you made yourself.

SEGMENT 5 - 25 MIN

Prove it, break it, reset and write up

Both tiers

Prove it, then break it

PROVE IT

- Your signature verifies with your **public** key
- It fails after one edit of the file
- Plain web shows the index; secure web does not

✕ BREAK IT

- Change the last character of the signed file by one
- `Verification failure`: the digest changed completely
- Put it back; it verifies again

Reset, and the rules

RESET

- Copy your evidence first; delete your private key file
- Full tier: restore the `day-4-start` snapshot

RULES OF ENGAGEMENT

- No tool in this lab can affect anyone else's system
- You inspect only your own and public certificates, and capture only your own traffic

Record your evidence

EVIDENCE

- Each value goes in the template's table, with the step it comes from
- Never put your private key in the pack
- Your signed file, your public key, and the successful verification
- The certificate fields you read: subject, issuer, validity
- The three levels of a live certificate chain
- A note of what was readable over plain web and not over secure web
- The hand-drawn sign-and-verify sketch
- Your 120-word explanation

Questions & Discussion

Next: Day 5, who you are and what you may do.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision 1debf15